

# Week 1, Day 1 — presenter notes

Slides: open [week-01-day-01.html](#) in Chrome/Firefox → **F** fullscreen → arrows. **N** shows the speaker note for the current slide. **P** prints to PDF.

This session is **ideas + toolchain**. No SMS code. No slices. No REST.

## Timing (3–3.5 hours)

| Clock     | Block                                                   | Slides |
|-----------|---------------------------------------------------------|--------|
| 0:00–0:10 | Welcome, course promise, outcomes                       | 1–6    |
| 0:10–0:35 | Software engineering + why Go                           | 7–12   |
| 0:35–0:50 | Pinned tools, GOPATH vs modules                         | 13–15  |
| 0:50–1:40 | <b>Live</b> install Go + VS Code + format on save       | 14–17  |
| 1:40–2:20 | <b>Live</b> first program, export rule, run/build/gofmt | 18–20  |
| 2:20–3:00 | <b>Live</b> Git + GitHub                                | 21–23  |
| 3:00–end  | Lab checklist; collect GitHub URLs                      | 24–27  |

If install overruns, cut stories on Why Go. **Never cut** `gofmt` or the **GitHub push**.

## Live demo script

### 1. Install (Ubuntu)

Use the **exact** filename from <https://go.dev/dl/> on the day you teach (patch version may be 1.25.1, etc.). Do **not** `apt install golang-go`.

```
go version    # must say go1.25.x linux/amd64
go env GOROOT GOPATH
```

Windows students: Ubuntu app via **WSL2**, then the same commands. macOS: pkg from go.dev or Homebrew `go`, still check `go version`.

PATH pitfall: new terminal after editing `.bashrc`.

### 2. VS Code

1. Extension **Go** (Go Team at Google).

2. Command Palette → **Go: Install/Update Tools** → all.
3. User settings JSON → format on save (slide).
4. Ugly `hello.go` → save → it reformats. Applause optional.

### 3. First program

Type with them. File: `~/epic-go/hello/hello.go`.

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Epic Learn")
}
```

Show `fmt.Println` failing. Then:

```
gofmt -w hello.go
go run hello.go
go build -o hello hello.go
./hello
```

Do **not** `go build` today — there is no `go.mod` until week 4.

### 4. Git

```
git init
# .gitignore should include the binary `hello`
git add hello.go .gitignore
git commit -m "Add Hello Epic Learn"
```

Create empty GitHub repo → `remote add` → `push`. Collect the HTTPS URL.

### Lab gate (they do not leave without)

- `go version` is 1.25.x
- Format on save works
- `go run` and `./hello` both print the message
- GitHub has `hello.go`, not the binary
- You have their repo URL

## Homework

- `why-go.md` — half page, own words, why Go for SMS
- Screenshot of `go version`

## Typical failures

| Symptom                                  | Fix                                      |
|------------------------------------------|------------------------------------------|
| <code>go: command not found</code>       | PATH / new terminal / WSL vs PowerShell  |
| <code>go version</code> is 1.18 from apt | Remove distro Go; install from go.dev    |
| Extension does nothing                   | Wrong folder opened; tools not installed |
| <code>undefined: fmt.Println</code>      | Capital <b>P</b> — teaching moment       |
| Push rejected                            | Token, 2FA, or remote README conflict    |
| Huge file on GitHub                      | They committed <code>hello</code> binary |

## What not to teach today

Modules (`go mod init`), packages, functions as a topic, REST, Gin, PostgreSQL, the SMS repo. Tease SMS only.